FCLOUDE OS

+ KAYT AI

MASTER BLUEPRINT • VERSION 1.0

A practical, beginner-safe plan for building a customized Linux experience and a permission-controlled AI assistant — without trying to invent an entire operating system on day one.

Purpose: one source of truth for research in Gemini, architecture planning in Claude Code, and the founder's own learning.

Prepared as a working project document • 17 October 2026

This is a planning brief, not a security certification or a claim that the proposed system is already feasible in every detail.

How to use this document

Treat this PDF as the project's shared brief. Gemini should use it to research and challenge assumptions; Claude Code should use it to draft architecture and implementation plans only after the decisions are reviewed. Do not ask either AI to build the whole product in one shot.

The core recommendation

Build the assistant and the operating-system customization as two separate projects first. Start with a tiny KAYT prototype that runs as a normal user application. Separately learn to customize an existing Linux distribution in a virtual machine. Integrate them only after each part is understandable and testable.

The skyscraper rule

The first milestone is not "a custom OS." It is: **the founder can explain the architecture, run the tests, identify a failure, revert a change, and demonstrate that KAYT cannot perform an action outside its explicit permissions.** That is a real first brick.

Executive decisions (proposed defaults)

Decision	Proposed default	Why
Base system	Customize an existing Ubuntu/Debian-based desktop in a VM; do not write a kernel.	Keeps the project focused on the user experience and KAYT integration.
First KAYT version	Local CLI or minimal desktop app; explicit user-provided input only.	Small enough to understand and test.
AI execution	AI proposes; deterministic code checks permissions; user confirms actions.	The model must not be its own security boundary.
Development	Git + Python + a code editor + VM snapshots.	Low cost, reversible, and beginner-friendly.
Distribution packaging	Try Cubic for an Ubuntu/Debian-based ISO after basic Linux learning.	Avoids creating a distribution build pipeline too early.
Budget	Zero-cost tools first; avoid paid API commitments until a useful prototype exists.	Protects the project from unnecessary recurring costs.
Scope	One user problem, one reliable workflow, one demonstrable prototype.	Prevents feature sprawl.

Important: a customized ISO is not automatically a new independent distribution. A polished wallpaper and bundled app are customization; a dependable OS product also needs update strategy, package provenance, recovery, documentation, compatibility testing, and long-term maintenance.

Contents

Use the section numbers when discussing changes so research notes, code plans, and decisions remain traceable.

1. Product definition and non-goals
2. Product architecture: the layers and their boundaries
3. What to build yourself vs reuse
4. Recommended free / open-source tool stack
5. KAYT MVP: features and exclusions
6. Security and permission model
7. Linux customization strategy
8. Beginner learning ladder and roadmap
9. Repository structure and engineering workflow
10. Testing, release gates, and success metrics
11. Risks, costs, and maintenance reality
12. Research checklist for Gemini
13. Architecture prompt for Claude Code
14. Daily AI workflow: how to use assistants without outsourcing understanding
15. Glossary
16. Official reference links

1. Product definition and non-goals

Working concept

FCLOUDE OS is a proposed, security-conscious, AI-friendly desktop experience based on an established Linux distribution. **KAYT** is the assistant that helps a user understand information and perform carefully scoped tasks through a transparent, permission-controlled interface.

Problem statement to validate

People may want a desktop assistant that can help with selected local tasks without receiving unrestricted access to the computer. The product should make it easy to see what KAYT can access, what it is proposing, what it has done, and how to stop or undo supported actions.

Do not assume this problem is proven. Research existing products and interview a few likely users before investing heavily.

Proposed product principles

- **User control:** the user can inspect, approve, cancel, and revoke permissions.
- **Least privilege:** KAYT receives only the minimum access required for the current task.
- **Transparency:** show the planned action, target, reason, and result.
- **Local-first where practical:** do not upload files or send data to a remote model unless the user has been told and the feature requires it.
- **Safe failure:** if the model, network, or a tool fails, do not improvise risky actions.
- **Accessible and useful:** solve a real, narrow problem before adding impressive-looking features.
- **Maintainability:** prefer upstream components with active maintenance over custom replacements.

Non-goals for version 0.1

- No custom kernel, compiler, programming language, package manager, or bootloader.
- No autonomous administrator/root access for KAYT.
- No face-recognition login or biometric feature in the first prototype.
- No always-listening microphone, covert monitoring, or broad background surveillance.
- No automatic deletion, installation, network configuration, or shell command execution without a separate, explicit permission design.
- No promise that AI output is always correct, private, or secure.
- No mobile ecosystem, app store, cloud sync platform, subscription bundle, or unrelated startup features in the MVP.

Definition of the first useful result

A small KAYT program accepts a user-selected text input, summarizes or explains it, and produces a visible result. It cannot roam the filesystem, run shell commands, or elevate privileges. The founder can explain the input path, model interface, permission check, error handling, and tests.

2. Product architecture: layers and boundaries

Keep the system modular. The desktop shell, assistant, permission checks, operating system, and model provider should not become one tightly coupled blob.

Layer	Responsibility	Initial implementation
User experience	Launcher, settings, status, conversation view, approval prompts, history.	Use the existing desktop environment; build a minimal KAYT interface later.
KAYT application	Conversation state, task planning, input/output, user-visible explanations.	Python application with a small, clearly defined interface.
Tool gateway	Exposes a small set of allowed operations to KAYT.	A hand-written allowlist; no generic unrestricted command runner.
Policy / permission layer	Checks who requested an action, which resource is targeted, and whether approval is required.	Deterministic code outside the model; default deny.
Audit log	Records approved actions, result, time, and failures without unnecessarily storing sensitive content.	Local append-only style event log for prototype; define retention policy.
Model adapter	Connects to a local model or a remote API behind one interface.	Start with a mock provider; add one real provider only after tests.
OS integration	Packages KAYT, launches it, applies desktop settings, handles updates.	Existing Linux distribution and packaging tools.
Linux base	Kernel, user accounts, permissions, filesystem, services, drivers, updates.	Upstream supported Linux base; no custom kernel initially.

Trust boundary

Treat model output as untrusted text. The model may misunderstand a request, be manipulated by hostile text inside a document, or propose a harmful action. It must never be allowed to rewrite its own permissions or skip the policy layer.

Safe request flow

1. User chooses a task and explicitly selects the input resource.
2. KAYT produces a proposed plan in structured data.
3. Application validates the plan against a strict schema.
4. Policy layer checks the allowlist and resource boundaries.
5. If the action changes state or has meaningful side effects, show a human-readable confirmation screen.
6. Execute only the approved, narrow operation through a dedicated tool function.
7. Record the action outcome and surface errors honestly.
8. Provide an undo path where feasible; otherwise clearly state that the action is not reversible.

Do not confuse a UI confirmation button with a complete security architecture. The backend must re-check permissions at the moment of execution.

3. What to build yourself vs reuse

A strong product does not need every component to be invented from scratch. Your distinctive work should be the parts that make the experience safer, clearer, and more useful.

Component	Decision	Reason
Linux kernel and drivers	REUSE upstream	Enormous maintenance and hardware compatibility burden; no unique value for the first product.
Desktop environment / window manager	REUSE first	Custom desktop shell is a separate project; start with configuration and a small app.
Base packages and update system	REUSE upstream	Security updates and dependency management are critical ongoing responsibilities.
ISO customization/build pipeline	REUSE a tool first	Learn packaging before maintaining a custom build system.
KAYT workflow and user experience	BUILD incrementally	This is central to the product concept.
Permission policy and tool gateway	BUILD narrowly, test heavily	Core differentiator: model suggestions must not become unrestricted machine access.
Action history and approval UX	BUILD a minimal version	Makes actions visible and controllable.
Model runtime/provider	ADAPTER, don't reinvent	Model training and inference infrastructure are not the first milestone.
Identity/authentication	REUSE OS accounts first	Biometrics add privacy, security, and support complexity.
Telemetry/analytics	OMIT initially	Collecting data adds privacy obligations and is not needed to prove the concept.
Auto-updater	REUSE upstream first	A custom updater can break machines if it mishandles signatures, rollback, or failures.
Custom language/compiler	OMIT	No clear product requirement; would distract from KAYT and the desktop experience.

The “mine” test

Build a component yourself only if at least one is true: (a) it creates the user-visible difference, (b) it is necessary to enforce your product’s permission model, or (c) implementing a tiny version is the best way to learn a prerequisite. Otherwise, evaluate a maintained existing component.

Before adopting a dependency, record: license, maintainer/activity, release cadence, security advisories, platform support, dependency footprint, data sent off-device, migration cost, and how to remove or replace it.

4. Recommended free / open-source tool stack

“Free” can mean no payment, open-source, or a free tier with limits. These are different. Verify each tool’s current licensing, terms, privacy policy, quotas, and availability before depending on it.

Need	Recommended starting choice	Alternatives / notes
Host OS	Keep Windows 11 for now.	Avoid risking your main laptop installation while learning.
Linux test system	Ubuntu Desktop or another established Ubuntu/Debian-based desktop in VirtualBox.	Use a VM snapshot before experiments. The base choice can be revisited after research.
Virtualization	VirtualBox (free for core use; check extension-pack terms separately).	QEMU/KVM is a later alternative, especially on a Linux host.
Editor	VS Code core editor or VSCodium.	VS Code's core editor is free; its distribution/licensing differs from the fully open-source Code-OSS build.
Language	Python for the first KAYT prototype.	Avoid introducing Rust, C++, TypeScript, and shell scripting all at once.
Version control	Git + a private GitHub repository.	Git itself is free/open source; GitHub Free has plan limits and terms.
AI-assisted coding	Use the AI tools you already have access to; try Continue if you want an open-source editor extension.	AI extension may be open-source while model inference is not free; model/API cost depends on provider.
Local model experiment	Ollama, only if hardware performance is acceptable.	Local inference can be slower and uses RAM/disk; do not assume the laptop can run large models well.
ISO customization	Cubic for a later Ubuntu/Debian-based live ISO experiment.	It is a customization tool, not a substitute for release engineering.
Documentation	Markdown files in Git; optionally MkDocs later.	Avoid a separate documentation platform until needed.
Task tracking	GitHub Issues/Projects or a plain Markdown TODO.	Do not spend weeks designing a productivity system.
Design mockups	Pen-and-paper or diagrams.net.	Figma free tier is an option but is not required for MVP.
Testing	Python unittest/pytest, manual VM test checklist.	Keep tests close to the code; no need for complex CI at first.

Recommended minimal stack for the first 30 days

VirtualBox + one Linux VM + Python + Git + a code editor + Markdown notes. Add one AI coding tool if it genuinely helps. Do not install five agents, three model runtimes, multiple distro builders, and a container platform before the first working program exists.

Hardware reality

The current laptop has 16 GB RAM and integrated graphics. Start with modest VM allocation and close heavy apps. Because previous virtual machines felt slow, begin with a lightweight CLI project and only one Linux VM at a time. Local AI models may be possible but are not guaranteed to feel responsive; use a mock model or a carefully budgeted remote API for the first interface test.

5. KAYT MVP: features and exclusions

Version 0.1 should prove one narrow workflow end-to-end. A reliable, understandable small assistant is more valuable than a broad agent that occasionally does the wrong thing.

MVP workflow

1. User opens KAYT.
2. User types a question or selects one text file using a normal file picker.
3. KAYT explains what input it will use and whether a remote service will receive it.
4. KAYT returns a summary or explanation with clear error messages.
5. The app shows the provider used and offers a way to clear the current session.

Feature priority

Priority	Feature	Acceptance condition
P0	Text input and output	A user can enter text and get a response from a mock provider.
P0	Selected-file input	Only a user-selected supported text file is read.
P0	Provider adapter	Model provider can be swapped without rewriting the UI or permission logic.
P0	Input-size limit and errors	Oversized/unsupported input fails clearly and safely.
P0	Permission allowlist	The app cannot access arbitrary paths or invoke a shell through the model.
P1	Action history	Shows task, approval status, result status, and timestamp without storing unnecessary sensitive text.
P1	Explicit remote-data notice	Before remote processing, user is told what content is sent.
P2	Minimal desktop UI	A simple window or tray entry after CLI behavior is stable.
Later	Approved file organization or system actions	Only after threat modelling, narrow tool design, confirmation, tests, and rollback.
Out of scope	Autonomous terminal, admin access, arbitrary file search	Do not add in MVP.

Do not build these yet

- Voice assistant, wake word, always-on listening.
- General web browser automation or autonomous internet research.
- Automatic software installation and system settings changes.
- Face recognition, biometric authentication, or identity inference.
- Cloud account sync, telemetry, or a multi-agent office.
- A custom desktop shell, package manager, kernel, compiler, or programming language.

6. Security and permission model

Security is not a slogan or a single feature. It is a collection of explicit boundaries, small privileges, careful defaults, testing, and maintenance. This section is a starting design, not a guarantee of security.

Threats to consider

Threat	Example	Initial mitigation
Prompt injection	Text inside a selected document tells KAYT to ignore instructions and exfiltrate data.	Treat document content as untrusted; no direct tool authority from document text; user approval and policy checks.
Over-broad file access	KAYT reads unrelated files or secrets.	File picker, canonical path checks, allowlisted file types, size limits, no recursive filesystem search.
Command execution	Model constructs a shell command that deletes files or downloads/runs code.	No generic shell tool in MVP; dedicated functions only.
Privilege escalation	KAYT requests administrator rights.	Run as ordinary user; never give the model sudo/root credentials.
Remote data exposure	Private document is sent to a model provider.	Local mock by default; clear notice and consent before remote processing; minimize transmitted content.
Malicious dependency	Package installation adds compromised code.	Use trusted sources, pin versions where appropriate, review dependencies, keep dependency list small.
Secrets leakage	API key is committed to Git or included in prompts.	Environment variables or local ignored config; secret scanning; never commit keys.
Unsafe update	An update breaks or compromises the app.	Use upstream OS update mechanisms; document versions; test in clean VM before distribution.

Non-negotiable design rules

- **Default deny:** every tool operation must be explicitly allowed.
- **Separate policy from the model:** permission decisions live in deterministic application code.
- **Least privilege:** run as the logged-in user, not administrator.
- **Human confirmation:** required for any meaningful state-changing action; confirmation must describe the target and consequence.
- **Constrained tools:** provide functions such as read_selected_text_file(), not run_any_shell_command().
- **Validate everything:** validate model output, file type, size, path, and tool arguments.
- **Audit carefully:** log events without casually logging passwords, API keys, or entire private documents.
- **Fail closed:** if permission validation fails, do not execute the action.
- **Recovery:** keep backups and test rollback before experiments.

Prototype tests that must pass

- KAYT cannot read a file that the user did not select.
- KAYT cannot call an arbitrary shell command because no such tool exists.
- A malformed model response is rejected.
- A file outside the permitted scope is rejected even if the model asks for it.
- Cancel means no action occurs.
- Remote processing is not used silently.
- No API key or personal document content appears in Git history or ordinary logs.
- An exception does not bypass policy checks.

Important: no single checklist makes an AI agent secure. The design needs adversarial testing and independent review before anyone relies on it for sensitive tasks.

7. Linux customization strategy

Do not start by building Linux from source. First learn how a distribution is assembled, configured, updated, and tested. A custom live ISO is a packaging milestone, not the first programming milestone.

Recommended progression

Stage	Deliverable	What it teaches
A. Learn an existing distro	A Linux VM you can use comfortably.	Filesystem, users, permissions, processes, packages, services, updates.
B. Configure the desktop	A reproducible list of settings and installed apps.	Desktop environment, configuration, dependencies.
C. Package KAYT as an app	KAYT launches on the VM as a normal user.	Packaging, entry points, dependencies, logs.
D. Create a customized ISO	Bootable test image in a VM.	ISO customization, boot testing, package inclusion.
E. Rebuild reproducibly	Documented steps to create the same image again.	Versioning, repeatability, release process.
F. Test on clean VM	Install/boot test report with issues recorded.	Compatibility, first-run experience, recovery.
G. Limited public preview	Small group of voluntary testers and feedback.	Support, documentation, real-world reliability.

Tool choice

Cubic is a practical candidate for customizing an Ubuntu/Debian-based live ISO through a guided workflow. Verify its current supported host distributions and target ISO compatibility before using it. The official project page describes it as a GUI wizard for creating customized Ubuntu/Debian-based live ISOs.

Cubic official project

Linux From Scratch is valuable educational material for understanding how a Linux system is built from source, but it is not the recommended first path to a usable product. Save it for later study if the underlying systems knowledge becomes a genuine goal.

Linux From Scratch

Distro choice principles

- Prefer a maintained base with clear security updates and broad hardware support.
- Avoid choosing a distro solely because it looks advanced or is popular among hackers.
- Do not replace the package manager or update mechanism in the first release.
- Document every customization as a script or config where possible; avoid undocumented manual edits.
- Test installation, boot, network, sound, suspend/resume, display scaling, and updates in a clean VM before distributing an ISO.
- Do not install a prototype directly over the Windows host or repartition the laptop for this experiment.

8. Beginner learning ladder and roadmap

Use this as a sequence, not a deadline contract. If a stage takes longer, that is normal. A stage is complete when you can demonstrate and explain it—not when a calendar says it should be done.

Phase	Focus	Deliverable / gate
0 — Setup	Create project folder/repository, notes, and a safe VM; take a snapshot.	Can restore the VM and locate the project files.
1 — Linux basics	pwd, ls, cd, mkdir, cp, mv, cat, grep, find, chmod, ps, top, package management, users, permissions, services.	Can explain where files live, how permissions work, and how to inspect a process.
2 — Git + Python	Variables, functions, files, exceptions, modules, JSON, virtual environments, tests; git status/add/commit/diff/log.	A tiny tested Python program committed to Git.
3 — KAYT mock	Input/output, mock model adapter, selected text input, error handling.	Works without a paid model API.
4 — Permission boundary	Allowlisted tool functions, explicit input, path checks, structured response validation.	Unauthorized access tests fail safely.
5 — One model adapter	Add one local or remote provider behind an interface.	Provider can be changed without rewriting policy code.
6 — Minimal interface	Basic desktop window or simple terminal interface, user-visible status and errors.	A user can complete the narrow workflow without reading source code.
7 — Linux integration	Package KAYT, launch it in a VM, record dependencies.	Installs and launches as a normal user.
8 — Custom ISO	Cubic or another appropriate builder, test image, clean VM tests.	Bootable prototype plus a reproducible build note.
9 — Feedback	Ask a small number of willing users to try one task; collect issues and consent-based feedback.	Evidence about usefulness, not just compliments.

Suggested weekly rhythm

Start with 4–6 focused hours per week if that fits around school, sleep, and family responsibilities. For example: two short learning sessions, one building session, and one review/test session. If exams or family needs increase, reduce project time rather than sacrificing sleep.

What to learn first (order matters)

1. Linux command line and filesystem concepts.
2. Git basics and how to inspect changes.
3. Python fundamentals and simple automated tests.
4. How applications read input, handle errors, and call APIs.
5. Threat modelling and permission checks.
6. Desktop packaging and Linux image customization.
7. Only then: deeper model orchestration, advanced OS internals, or custom desktop components.

9. Repository structure and engineering workflow

Keep the first repository small and understandable. Split repositories later only if there is a clear reason; premature multi-repo architecture can create overhead.

Suggested early repository layout

```
fcloude-kayt/\n README.md\n LICENSE\n SECURITY.md\n docs/\n product-brief.md\n architecture.md\n
threat-model.md\n decisions/\n src/\n kayt/\n app.py\n providers/\n tools/\n policy/\n logging/\n tests/\n
pyproject.toml\n .gitignore\n .env.example
```

Do not create empty modules just to match this tree. Start with the smallest layout that fits the first testable feature and add folders when the code needs them.

Every change should follow this loop

1. Write the requirement in one sentence and define a testable acceptance condition.
2. Ask the AI for a design explanation and likely risks before asking it to write code.
3. Ask for a small patch—not a whole application rewrite.
4. Review the diff and ask what each changed file is responsible for.
5. Run the tests in the VM or a safe development environment.
6. Manually test both success and failure paths.
7. Commit a working checkpoint with a clear message.
8. Update documentation if behavior, permissions, setup, or limitations changed.

Version-control rules

- Commit only known-good, reviewable changes.
- Never commit API keys, passwords, private documents, VM disk images, or large model files.
- Use a private repository while experimenting with secrets or unreleased implementation details.
- Keep a changelog or short decision log for major architecture choices.
- When an AI proposes a change you cannot explain, pause and learn before merging it.

10. Testing, release gates, and success metrics

A demo that works once is not a release. Every phase needs a gate that blocks premature expansion.

Gate	Must be true before moving forward
Gate 1: learning setup	VM snapshot/restore works; project is version-controlled; basic commands are understood.
Gate 2: first program	A small Python program runs, has basic tests, and its author can explain each function.
Gate 3: KAYT mock	The user can provide text and get a predictable mock response; invalid input produces a safe error.
Gate 4: permissions	Unauthorized path/tool tests fail; no unrestricted shell access exists.
Gate 5: model provider	Provider failure, malformed output, and network failure are handled without bypassing policy.
Gate 6: desktop integration	KAYT runs without administrator rights; uninstall/reinstall procedure is documented.
Gate 7: custom ISO	Image boots in a clean VM; KAYT launches; updates and recovery have been considered.
Gate 8: outside testers	Limitations and data handling are disclosed; feedback can be reported; no sensitive real-world task is relied on.

Useful measurements

- Task completion: can a user complete the one chosen workflow?
- Correctness: how often does KAYT produce a useful, verifiable result?
- Permission correctness: do forbidden actions remain blocked under adversarial inputs?
- Recovery: can the user recover from a failed task or broken test image?
- Comprehension: can the founder explain the code and architecture without relying on AI narration?
- User value: do testers voluntarily return to use it for the same task?
- Maintenance: how much effort does a clean rebuild and update test require?

Do not measure success mainly by GitHub stars, visual polish, number of agents, or lines of code. Measure whether the prototype is useful, understandable, and safe enough for its stated scope.

11. Risks, costs, and maintenance reality

Risk	Why it matters	Control
Scope explosion	A distro, AI agent, shell, security framework, and model runtime are separate complex projects.	Keep the MVP to one workflow; use a written non-goals list.
AI-generated code is misunderstood	The app may contain insecure or broken assumptions.	Small patches, code review, tests, and founder explanation gate.
Local model performance	Integrated graphics and 16 GB RAM may limit speed/size.	Start with mock provider; benchmark before choosing local models.
Remote model cost/privacy	APIs can cost money and transmit content.	Explicit provider notice, minimal data, spending limits, no secrets in prompts.
Distro maintenance	Base packages and security updates keep changing.	Stay close to upstream and define update responsibilities early.
Compatibility	Wi-Fi, graphics, suspend, and installers vary across hardware.	Test in VM first and then carefully on separate non-critical hardware if available.
Security overclaim	A few checks do not make an OS secure.	Use precise language, document threat model, request independent review before public reliance.
Founder burnout / school conflict	Overcommitment can make a long project unsustainable.	Small weekly schedule, breaks, and exam-period flexibility.

Cost policy

- Keep the first milestone at ■0 if possible.
- Do not buy hardware, cloud credits, domains, or subscriptions until a real bottleneck is measured.
- If using a remote API, set a hard spending cap or use mock/local inference for development.
- Track each paid dependency and what it provides; remove tools that are not being used.
- Use only software and models whose licenses permit the intended use; verify this before distribution.

Licensing reminder

Open source does not mean “no license obligations.” Check the licenses for the base distribution, bundled apps, icons, fonts, models, and any code copied into the project. Preserve notices and comply with redistribution requirements. Do not claim ownership of upstream components.

12. Research checklist for Gemini

Attach this PDF to Gemini, then paste the prompt below. Ask it to return a research report with source links and dates. Treat its output as research to verify, not as automatic truth.

```
You are a skeptical technical researcher reviewing the attached FCLOUDE OS + KAYT Master Blueprint v1.0.

Do not simply agree with the concept. Research current official documentation and reputable technical
sources, and clearly separate verified facts, reasonable assumptions, and unresolved questions.

Research tasks:
1. Identify the closest existing products/projects to the proposed security-conscious AI desktop assistant
and customized Linux desktop. Compare their actual capabilities, limitations, licenses, maintenance
activity, privacy posture, and likely overlap. Do not assume novelty.
2. Validate the proposed architecture: existing Linux base, separate KAYT app, deterministic
permission/policy layer, narrow tool gateway, model adapter, action history, and later ISO customization.
3. Compare appropriate Ubuntu/Debian-based customization/build options (including Cubic and official
distribution tooling). State host/target compatibility, current maintenance status, limitations,
reproducibility, and licensing. Prefer official project documentation.
4. Compare free/open-source options for Python development, Git hosting, testing, local inference, AI
coding assistance, documentation, issue tracking, and UI mockups. Distinguish genuinely free/open-source
tools from free tiers with limits and from paid model inference.
5. Threat-model KAYT: prompt injection through selected files, over-broad file access, command execution,
privilege escalation, secret leakage, remote data transfer, unsafe dependencies, and update compromise.
Propose tests and mitigations. Do not claim any design is secure without evidence.
6. Identify the smallest valuable MVP that can be built by a beginner, and list features that should be
postponed or cut.
7. Estimate likely hardware, storage, RAM, time, and cost constraints for a Windows 11 laptop with 16 GB
RAM, integrated graphics, and VirtualBox. State uncertainty and avoid assuming large local models will
perform well.
8. Provide a 30-day learning plan and a 90-day validation plan with measurable gates that preserve school
and health.
9. Flag licensing, trademarks, redistribution, privacy, security-update, and support obligations.
10. Challenge the product thesis: what user problem is being solved, who has it, and what evidence would
disprove the idea?

Output format:
A. Executive verdict (continue / narrow / rethink, with reasons).
B. Top 10 corrections to the blueprint.
C. Competitor/alternative table with primary-source links and evidence date.
D. Recommended free tool stack with costs/limitations.
E. Revised architecture and threat model.
F. MVP acceptance criteria and explicit non-goals.
G. 30-day plan and 90-day validation gates.
H. Unanswered questions ranked by risk.
I. Sources list with dates, prioritizing official docs and active repositories.

Rules:
- Do not generate implementation code yet.
- Do not recommend replacing the kernel or writing a programming language for the MVP.
- Do not assume the AI should have unrestricted computer access.
- If a source is unclear or stale, say so and request verification.
- Make recommendations specific, economical, and beginner-friendly.
```

How to evaluate the research

- Prefer official docs, active repositories, release notes, security advisories, and licenses.
- Check that claims are current and match the actual version being discussed.
- Require evidence for claims such as “secure,” “free,” “private,” “maintained,” and “best.”
- Reject long lists of tools unless each has a role in the MVP.
- Ask follow-up research only for unresolved decisions; do not restart broad research every time.

13. Architecture prompt for Claude Code

Use this only after reviewing Gemini’s research and deciding which recommendations to accept. In Claude Code, ask for architecture first and explicitly tell it not to write the full application until the plan is approved.

```
You are acting as a careful software architect and mentor for a beginner building FCLOUDE OS + KAYT. The
attached PDF is the current product brief. Read it fully before proposing code.

Your task in this phase is architecture and a tiny implementation plan, not a complete product.

Project constraints:
- The founder is a beginner. Every component must be explainable and testable.
- Keep the first KAYT version separate from the Linux distribution customization.
- Use Python for the first small prototype unless you provide a concrete reason not to.
- Start with a mock model provider. Do not require paid APIs to pass the first tests.
- Input must be typed by the user or a single text file explicitly selected by the user.
- No unrestricted filesystem traversal, shell execution, administrator/root privileges, autonomous
installs, or background surveillance.
- Model output is untrusted. Permission enforcement must live in deterministic code outside the model.
- Keep dependencies minimal and explain each one.
- Use a safe development environment; do not alter host partitions or replace the host OS.
- Do not put secrets in source code, Git, logs, or prompts.
- Do not implement extra features that are not in the MVP.

First response must contain:
1. Your understanding of the product in 5–10 bullets.
2. The smallest architecture that meets the MVP.
3. A data-flow diagram in plain text.
4. A list of modules/files and the responsibility of each.
5. A threat model covering prompt injection, path access, malformed model output, remote data transfer, and
secret leakage.
6. Test cases, especially negative tests proving disallowed actions fail.
7. A staged implementation plan with one small deliverable per step.
8. Questions that genuinely block safe implementation; do not ask questions already answered in the PDF.
9. A list of decisions that should remain unresolved until research is reviewed.

Do not write the full application yet. Wait for the founder to approve the architecture.

When implementation is approved:
- Implement one step at a time, with a small patch.
- Before changing files, state the intended change and acceptance tests.
- Explain every new dependency and each non-obvious code section in beginner-friendly language.
- Show the test command and expected result.
- Include failure handling and how to revert the change.
- Never execute destructive commands or change the host OS.
- Never claim the app is secure simply because tests pass.
- Stop and ask for review if a feature requires broader permissions than the current MVP.
```

14. Daily AI workflow: use assistants without outsourcing understanding

AI can accelerate research, explain errors, suggest tests, and draft small patches. It cannot replace the founder's responsibility to understand, validate, and maintain the system.

Use this prompt pattern

```
"Explain the concept first in beginner-friendly terms. Then propose the smallest change that meets this requirement: [requirement]. List files to change, assumptions, risks, and tests. Do not edit anything yet. After I approve, make only that change. Explain the diff line by line and tell me how to verify and revert it."
```

The 6-step loop

1. **Understand:** ask what problem a component solves and what alternatives exist.
2. **Plan:** ask for a small design and tests before code.
3. **Implement:** request a narrow patch.
4. **Inspect:** read the diff; ask about every unfamiliar line.
5. **Verify:** run tests and try failure cases inside the VM.
6. **Own it:** write a short note explaining what changed, why, and what remains risky.

Red flags: stop and investigate

- An AI tells you to run a command with sudo/root but cannot explain every part.
- A command downloads a script and executes it directly without review.
- A suggested command can delete files, rewrite disks, change partitions, or overwrite configuration.
- A model asks you to paste passwords, tokens, private keys, or private documents into a prompt.
- The AI makes large changes across many files without a test plan.
- A proposed feature gives KAYT broader access just because it is easier to implement.

A founder's rule

If you cannot explain what the code does, why the permission is needed, how to test it, and how to undo the change, it is not ready to merge.

15. Glossary

Term	Plain-language meaning
Distribution / distro	A packaged operating system built around the Linux kernel plus tools, libraries, package management, and defaults.
Kernel	Core software managing processes, memory, hardware access, and other system resources.
Desktop environment	The graphical experience: panels, settings, windows, app launcher, and related components.
ISO image	A file that can contain a bootable installer or live operating system.
Live ISO	A bootable environment that can run without a full installation; persistence and install behavior vary.
Package manager	The system used to install, update, and remove software packages.
Upstream	The original project from which a distribution or component is derived.
MVP	Minimum viable product: the smallest version that can test whether a real user problem is being solved.
Adapter	A small interface that lets the app swap one provider or implementation for another.
Least privilege	Give a component only the access it needs, for only as long as needed.
Threat model	A structured description of what can go wrong, who/what might cause it, and what protections are needed.
Prompt injection	Untrusted content attempts to manipulate an AI system into ignoring its intended boundaries.
Allowlist	An explicit list of permitted operations or resources; anything else is denied.
Reproducible build	A build process that is documented and repeatable, ideally producing equivalent outputs from the same inputs.
Regression test	A test that helps ensure a previously working behavior does not break after a change.

16. Official reference links

These are starting points, not endorsements of every feature or a guarantee that terms and availability will remain unchanged. Verify the specific version, license, and compatibility before adopting a tool.

Topic	Reference	Why it is included
Ubuntu Desktop	Ubuntu download and product information	Candidate base for a beginner-friendly VM.
Cubic	Official Cubic project repository	GUI tool for customized Ubuntu/Debian-based live ISO images.
Linux From Scratch	Linux From Scratch project	Educational resource for later systems learning, not first product path.
Python	Python documentation	Language reference and learning material.
VS Code	VS Code getting started	Editor setup; core editor can be used without enabling AI features.
Continue	Continue documentation	Open-source AI coding assistant extension; inference/provider cost varies.
Ollama	Ollama Linux documentation	Local model runtime documentation; hardware and model license still matter.
VirtualBox	VirtualBox user documentation	VM snapshots and safe experimental workflow.
GitHub plans	GitHub plan documentation	Current free-plan limits should be checked before relying on a feature.

Research hygiene

- Prefer official project docs and repositories over blog summaries for technical claims.
- Check release dates, current support status, licenses, and security advisories.
- Do not assume a tool is free to use commercially just because the download is free.
- Re-check all links and facts when beginning implementation; this document is a versioned brief, not a live database.

Final operating principle

Learn the layer before you build the layer. Reuse commodity infrastructure. Build the differentiated experience and permission model. Test each brick before stacking the next.

End of FCLOUDE OS + KAYT Master Blueprint v1.0